

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Clase 4: Listas

Listas

Hasta ahora vimos **tipos básicos**: bool, int, float, string.

Las **listas** son secuencias de valores. Son un **tipo compuesto**.

```
[False, False, True, False]
```

```
[45, 657, 11, 0, 45, -303, 0, -11, 45]
```

```
[3.14159, 0.0]
```

```
['Marte', 'Venus', 'Júpiter']
```

Listas: operaciones básicas

```
# Creamos una lista con varios elementos.
mundiales = ['Corea-Japón', 'Alemania', 'Sudáfrica',
             'Brsail', 'Rusia']

# Leemos e imprimimos una posición de la lista.
print(mundiales[3])      # imprime Brsail

# Sobreescribimos una posición.
mundiales[3] = 'Brasil'

# Agregamos un elemento al final.
mundiales.append('Qatar')

# Imprimimos toda la lista.
print(mundiales)
```

Listas: operaciones básicas

Operaciones de listas:

```
a = []          # Crea una lista vacía.
a = [7, 1, 2]   # Crea una lista no vacía.
len(a)          # Devuelve la longitud.
a.append(10)     # Agrega un valor al final.
a[i]             # Lee el i-ésimo valor.
a[i] = 8         # Escribe en la i-ésima posición.
x in a           # Consulta pertenencia.
```

Listas: iteradores

La instrucción `for` nos permite `iterar` los elementos de una lista:

```
mundiales = ['Sudáfrica', 'Brasil', 'Rusia', 'Qatar']  
  
for m in mundiales:  
    print(m)
```

```
Sudáfrica  
Brasil  
Rusia  
Qatar
```

Listas: iteradores

La instrucción `for` nos permite `iterar` los elementos de una lista:

```
mundiales = ['Sudáfrica', 'Brasil', 'Rusia', 'Qatar']  
  
for m in mundiales:  
    print(m)
```

A medida que el ciclo se ejecuta, `m` es simplemente un nombre que usamos para referirnos a cada elemento de la lista. Podría reemplazarse por cualquier otro nombre sin que eso afecte el resultado.

Por ejemplo, el siguiente código es equivalente al anterior:

```
mundiales = ['Sudáfrica', 'Brasil', 'Rusia', 'Qatar']  
  
for pais in mundiales:  
    print(pais)
```

Listas: iteradores

La instrucción **for** nos permite **iterar** los elementos de una lista:

```
mundiales = ['Sudáfrica', 'Brasil', 'Rusia', 'Qatar']  
  
for m in mundiales:  
    print(m)
```

También es posible iterar sobre otros tipos de datos, como strings:

```
for c in 'Algoritmos':  
    print('*** ' + c + ' ***')
```

Y sobre conjuntos, diccionarios, archivos, entre otros. Más adelante volveremos sobre estos casos.

Funciones y parámetros

Ejercicio. (1)

- (a) Escribir en Python una función `sumar_todos(lista)` que dada una lista de enteros devuelva la suma de todos sus elementos.
- (b) Escribir en Python una función `sumar_positivos(lista)` que dada una lista de enteros devuelva la suma de todos sus elementos mayores que cero.

Listas: índices y sublistas

```
mundiales = ['Corea-Japón', 'Alemania', 'Sudáfrica',  
             'Brasil', 'Rusia', 'Qatar']  
len(mundiales) # devuelve 6  
mundiales[5]   # último elemento  
mundiales[6]   # error: índice fuera de rango  
mundiales[-1]  # último elemento!
```

Python permite obtener fácilmente **sublistas** (o *slices*):

```
mundiales[i:j] # sublista desde i hasta j-1  
mundiales[1:5] # del segundo al anteúltimo
```

El operador `[i:j]` es muy versátil. `i` y `j` pueden ser negativos, e incluso omitirse:

```
mundiales[2:]  # desde el tercer elemento hasta el final  
mundiales[:2]  # primeros dos elementos  
mundiales[-2:] # últimos dos elementos
```

Operaciones de listas y strings

Si tenemos una lista de strings y los queremos **unir** en un solo string, separados por coma (p.ej.), usamos **join**.

```
colores = ['verde', 'azul', 'rojo']  
'; '.join(colores)  
# devuelve 'verde; azul; rojo'
```

La operación inversa es **split**.

```
colores = 'verde; azul; rojo'  
colores.split('; ')  
# devuelve ['verde', 'azul', 'rojo']
```

Otras operaciones útiles de strings:

```
x = 'programación'  
'rama' in x      # True: 'rama' es substring  
'wxyz' in x      # False  
x[4:8]           # 'rama'
```

Listas y strings

Ejercicio. (2) Evaluar las siguientes expresiones, primero a mano, después revisar en la consola interactiva de IPython.

```
1 x = 'dificilísimo'
2 x.split('i')
3 x.split('i')[2:]
4 'i'.join(x.split('i')[2:])
5 'fa' + 'i'.join(x.split('i')[2:])
```

¿De qué tipo es cada expresión? ¿Qué valores tienen?

Ayudamemoria: Para conocer el tipo de una expresión `expr`, se puede usar la función `type(expr)`.

Repaso de la clase de hoy

- ▶ Listas:
 - ▶ Operadores
 - ▶ Iteradores
 - ▶ Sublistas

Bibliografía complementaria:

- ▶ IPP, secciones 9.1 a 9.7 y 9.16

Con lo visto, ya pueden resolver la sección 5 de la guía de ejercicios.